

100071011 Computer Networks 2025-2026-2
Project-2
Building a CGI-Support Multi-Threaded Web Server
Project Report

学号 (Student ID)	1120231863
姓名 (Name)	左逸龙
班号 (Class No.)	07112303
授课教师 (Instructor)	宿红毅

School of Computer
Beijing Institute of Technology
July 14, 2026

1. Requirement Analysis

1.1. Project Objective and Scope

The objective of this project is to build a small Web server directly on top of TCP sockets. The server must accept several clients at the same time, parse HTTP requests, return static files, and execute CGI programs for dynamic pages. It must also limit the number of open connections and keep one access-log entry for each request.

The implementation uses HTTP/1.0 as its main protocol version. GET, POST, and HEAD are required. GET retrieves a representation, POST submits form data for processing, and HEAD returns the same response headers as GET without an entity body [1]. A small HTTP/1.1 compatibility subset is also included because current browsers commonly send HTTP/1.1 requests.

The dynamic part contains two applications. The calculator receives two numbers and an operation. The query application receives a student ID and reads the matching name and class from a local database. Both applications run as separate CGI processes. According to CGI/1.1, the server passes request metadata through environment variables and passes a POST body through standard input [3].

1.2. Main Requirements

The main requirements and the corresponding parts of the implementation are listed in Table 1. The table also identifies the evidence used in later tests.

No.	Requirement	Implementation	Test evidence
1	Low-level Web server	TCP listening socket and direct HTTP message parsing.	Fragmented-request and end-to-end socket tests.
2	GET, POST, and HEAD	Method routing, form-body reading, and body suppression for HEAD.	Protocol and integration tests.
3	Static and dynamic pages	Safe file routing plus two CGI sub-processes.	Browser screenshots and CGI tests.
4	HTTP status codes	Custom pages for 400, 403, 404, and 500; 200 for successful requests.	Status-code integration tests.
5	Fixed thread pool	Workers are created during startup and wait for queued connections.	40-client concurrency test.
6	Maximum connections	The earliest accepted open connection is closed when the limit is reached.	Connection-manager and integration tests.
7	CGI data exchange	Environment variables, stdin, stdout, timeout, and output parsing.	Real-process CGI tests.
8	Calculator and query	Decimal arithmetic and a parameterized SQLite lookup.	Application tests and browser results.
9	Access log	A lock protects one Common Log style line per request.	Concurrent logging test.
10	Performance analysis	A repeatable local load tool varies the worker count.	10,500 measured requests.

Table 1: Traceability between the project requirements, implementation, and tests.

1.3. Constraints and Design Problems

The listener, connection limit, and worker count describe different resources. The connection limit bounds all open sockets, including connections waiting in the queue. The worker count determines how many connections can be processed at one time. When all workers are busy, additional accepted connections wait until a worker becomes available.

TCP provides a byte stream, so a complete HTTP request is not guaranteed to arrive in one `recv` call. The parser therefore needs a persistent buffer. It must find the header boundary, validate the request line and headers, and then read exactly the number of body bytes stated by `Content-Length`.

Serving a path supplied by a client also introduces a directory-traversal risk. A request such as `../../web_server.py` must not escape the document root. OWASP describes this attack as using path components to access files outside the intended Web root [8]. The server decodes the URL path strictly, resolves it, and verifies that the result remains under webroot before reading a file.

2. Design

2.1. Overall Architecture

The server is divided into networking, HTTP, application, CGI, data, and logging parts. The main thread owns the listening socket. Accepted connections enter a bounded connection manager, while a fixed set of workers removes connections from its queue. Each worker parses requests and calls the application router. The router either reads a static file or invokes an allowed CGI program. This structure is shown in Figure 1.

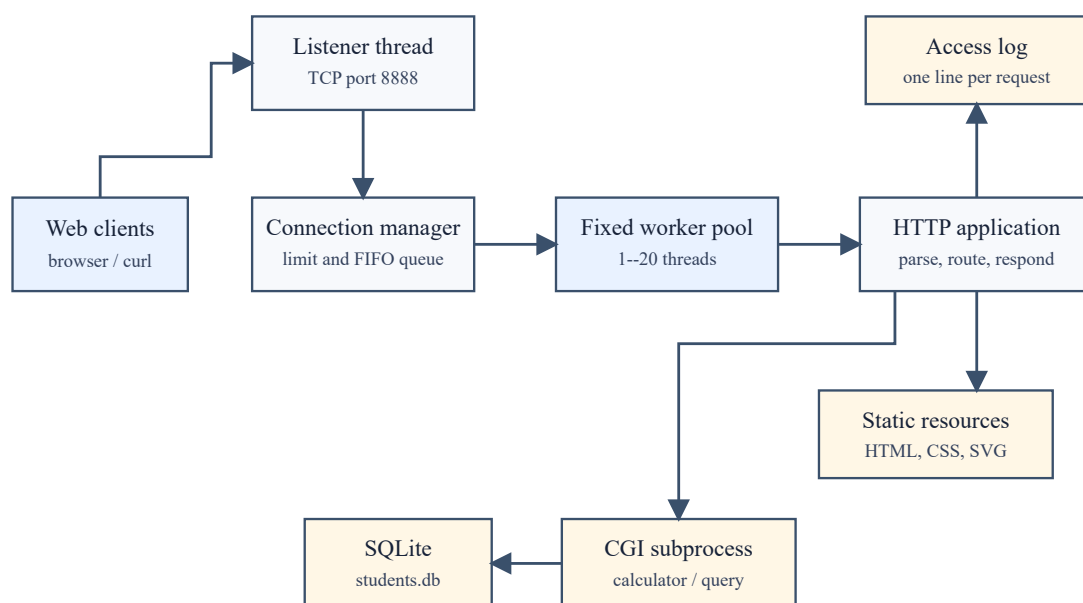


Figure 1: Overall structure of the multi-threaded Web server.

Python threads are suitable here because socket operations, file reads, and CGI waiting are I/O-bound activities. The standard threading module provides shared-memory concurrency, while `Condition` is used to put idle workers to sleep until a connection is queued [5].

2.2. Thread Pool and Connection Management

The server creates all worker threads before it begins normal service. A worker repeatedly calls `acquire()`, handles one connection, and finally releases it. This keeps the number of worker threads fixed throughout the run.

The connection manager stores open connections in an `OrderedDict` using acceptance order and stores pending work in a deque. One `Condition` protects both structures. If a new connection would exceed `max_connections`, the first entry in the ordered registry is removed. Its socket is shut down and closed, which wakes an active worker or prevents a queued connection from being acquired later.

Each worker can process several requests from the same socket. HTTP/1.0 closes after one response unless the client asks for keep-alive. HTTP/1.1 keeps the connection open by default unless the client asks to close it. A five-second idle timeout and a limit of 100 requests per connection prevent one client from occupying a worker indefinitely.

2.3. HTTP Message Processing

The request reader maintains a byte buffer for the whole connection. It first searches for CRLF CRLF, then parses the request line and headers. For POST, Content-Length is mandatory. The implementation rejects conflicting lengths, unsupported transfer encoding, headers above 64 KiB, and bodies above 1 MiB. These limits make malformed or unusually large requests fail with a controlled 400 response.

The response builder writes the status line and the Date, Server, Content-Length, and Connection headers. The entity body is appended for GET and POST. For HEAD, the builder retains the length of the corresponding representation but sends no body, which follows HTTP semantics [1][2].

Status	Meaning	Typical trigger	Response body
200	OK	Static file or successful CGI result.	Requested content.
400	Bad Request	Malformed request, invalid form input, or POST to a static page.	Explains that the request format or form input is invalid.
403	Forbidden	Traversal, protected directory, directory listing, or unlisted CGI.	Explains that the target path or CGI program cannot be accessed.
404	Not Found	Requested static file or CGI wrapper does not exist.	States that the requested resource does not exist.
500	Internal Server Error	CGI timeout, abnormal exit, malformed output, or internal failure.	Reports a CGI execution or internal processing failure.

Table 2: HTTP status codes generated by the server.

2.4. Static Resource Routing

The path / maps to index.html. Other static paths are decoded and resolved below the document root. Requests for directories, the data directory, the log directory, or CGI source files are rejected. The response MIME type is selected from the file extension, so HTML, CSS, and SVG resources are interpreted correctly by a browser.

The application layer separates routing from HTTP serialization. It returns an `HttpResponse` containing a status, headers, and bytes. The worker then serializes this object according to the request method and connection state. This separation also allows routing and serialization to be tested without starting the complete server.

2.5. CGI Processing

Only `/cgi-bin/calculator.py` and `/cgi-bin/query.py` are allowed. The CGI executor starts the selected program with `shell=False`, writes the request body to its standard input, and captures standard output and error. It supplies CGI/1.1 variables such as `REQUEST_METHOD`, `QUERY_STRING`, `CONTENT_LENGTH`, `CONTENT_TYPE`, `SCRIPT_NAME`, `SERVER_PROTOCOL`, and `REMOTE_ADDR` [3].

The child process must output CGI headers, a blank line, and the response body. The executor parses Status and Content-Type, removes hop-by-hop fields, and converts the result into an HTTP response. A process is terminated after five seconds, and output above 1 MiB is rejected. The data flow is shown in Figure 2.

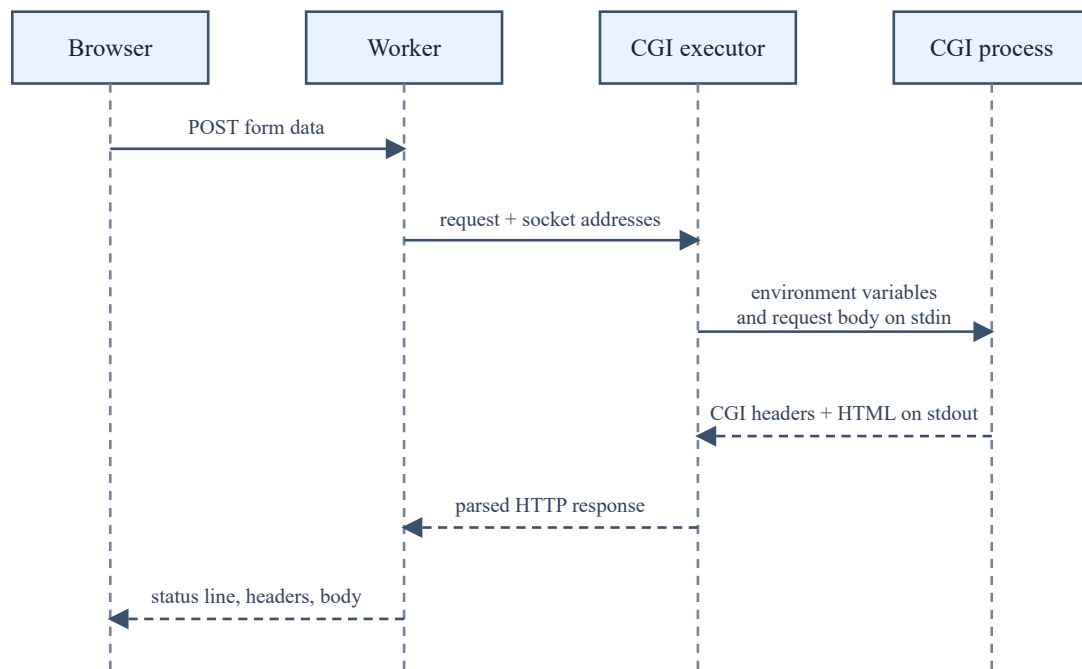


Figure 2: Data exchange for a POST request handled by a CGI process.

The calculator uses `Decimal` for the four arithmetic operations and checks invalid numbers and division by zero. The query program opens `SQLite` in read-only mode and uses a parameter placeholder in the SQL statement. User input is therefore passed as data instead of being joined into the SQL text [7]. At first startup, `students.db` is created atomically from a readable CSV seed.

2.6. Access Log and Shutdown

Every server run creates a timestamped log file. Each line records the client address, local time, request line, status, body size, referer, and user agent. Several workers can finish requests at the same time, so a lock protects each complete write. Quotes, backslashes, and newline characters from client headers are replaced before a line is written.

Shutdown sets a shared event, closes the listener, closes all registered client sockets, wakes workers waiting on the condition, and joins the worker threads. The log file is closed after the workers have stopped. This order prevents a worker from writing to a closed logger.

3. Development and Implementation

3.1. Languages and Tools

The development environment is listed in Table 3. The networking and HTTP layers use Python's low-level socket interface, whose methods correspond closely to the system socket calls [4].

Item	Version	Use in the project
Operating system	Windows 10 Home China 22H2 (64-bit, 19045.7417)	Development, browser tests, performance tests, and packaging.
Hardware	Lenovo 20QVA005CD; Intel Core i9-9880H (8 cores, 16 threads); 64 GiB RAM	Server execution, concurrent tests, and local performance experiments.
Command environment	PowerShell 7.6.3	Server startup, test execution, and auxiliary scripts.
Programming language	Python 3.13.3	Server, CGI programs, tests, and load tool.
Environment manager	uv 0.7.1	Creates and synchronizes the virtual environment.
Database	SQLite 3.49.1	Stores student records for the query CGI.
Test framework	unittest	Unit and end-to-end automatic tests.
Packaging	PyInstaller 6.21.0	Creates the Windows distribution directory.
Client tools	Microsoft Edge and curl.exe	Manual page and HTTP response checks.

Table 3: Development languages and tools.

3.2. Project Organization

Component	Responsibility
web_server.py	Command-line parsing, configuration, server startup, and frozen CGI dispatch.
server/core.py	Listener lifecycle, fixed workers, request loop, and graceful shutdown.
server/connections.py	Open-connection registry, pending queue, and oldest-connection eviction.
server/http.py	HTTP request buffering, parsing, keep-alive rules, and response serialization.
server/application.py	Static routing, protected paths, MIME types, CGI routing, and error pages.
server/cgi_executor.py	CGI environment, subprocess control, timeout, and output parsing.
server/access_log.py	Thread-safe access-log creation and writing.
cgi_apps/	Form parsing, calculator, database initialization, and student query.
webroot/	Pages, style sheet, SVG asset, CGI wrappers, data seed, and logs.
tests/ and tools/	Automatic tests, performance experiment, and auxiliary tools.

Table 4: Main source components and their responsibilities.

3.3. Selected Implementation Details

The worker pool is created once during startup. Each worker blocks on the connection manager when the queue is empty.

```
def _start_workers(self) -> None:
    for index in range(self.config.workers):
        thread = threading.Thread(
            target=self._worker_loop,
```

```

        name=f"web-worker-{index + 1}",
        daemon=False,
    )
    self._workers.append(thread)
    thread.start()

def _worker_loop(self) -> None:
    while True:
        context = self.manager.acquire()
        if context is None:
            return
        try:
            self._handle_connection(context)
        finally:
            self.manager.release(context)

```

Connection admission and eviction are performed while holding the same condition lock. This prevents a worker from receiving a queued connection at the same time that the listener removes it.

```

if len(self._connections) >= self.max_connections:
    _, evicted = self._connections.popitem(last=False)
    self._remove_pending(evicted.connection_id)
    evicted.closed.set()

context = ConnectionContext(
    connection_id=next(self._ids),
    socket=sock,
    address=address,
    accepted_at=time.monotonic(),
)
self._connections[context.connection_id] = context
self._pending.append(context)
self._condition.notify()

```

The request reader keeps unconsumed bytes for the next request on a persistent connection. After parsing the headers, it reads the body length stated by the client.

```

boundary = self._buffer.find(b"\r\n\r\n")
if boundary >= 0:
    header = bytes(self._buffer[:boundary])
    del self._buffer[:boundary + 4]
    return header

self._fill_buffer(content_length)
body = bytes(self._buffer[:content_length])
del self._buffer[:content_length]

```

The CGI process receives the request body through stdin. A timeout returns a controlled server error and also terminates the child process tree.

```

process = subprocess.Popen(
    command,
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    cwd=self.webroot,
    env=environment,
    shell=False,
)
try:
    stdout, stderr = process.communicate(
        input=request.body,
    )

```

```

        timeout=self.timeout,
    )
except subprocess.TimeoutExpired:
    self._terminate_process_tree(process)
    return self._error(500, "CGI program timed out")

```

The response serializer handles HEAD at the last stage. Earlier modules can still calculate the same representation and Content-Length as they do for GET.

```

head = ("\r\n".join(lines) + "\r\n\r\n").encode("iso-8859-1")
return head if request.method == "HEAD" else head + response.body

```

4. System Deployment, Startup, and Use

4.1. Running from Source

The source version requires Python 3.13 and uv. In PowerShell, open the source-project directory and synchronize the environment:

```
uv sync --dev
```

The server can then be started with its default settings:

```
.\.venv\Scripts\python.exe web_server.py
```

It listens on `http://127.0.0.1:8888/`, creates eight workers, and allows at most 32 open connections. The main options can be changed on the command line:

```

.\.venv\Scripts\python.exe web_server.py `
--host 0.0.0.0 `
--port 8888 `
--workers 4 `
--max-connections 32

```

Binding to `0.0.0.0` permits access through a local network if the Windows firewall allows the selected port. Pressing `Ctrl+C` starts the normal shutdown procedure.

4.2. Pages and Requests

URL path	Method	Function
/ or /index.html	GET/HEAD	Static home page.
/calculator.html	GET/HEAD	Calculator input form.
/query.html	GET/HEAD	Student query input form.
/cgi-bin/calculator.py	GET/POST	Dynamic arithmetic result.
/cgi-bin/query.py	GET/POST	Dynamic database query result.

Table 5: Main pages and CGI endpoints.

The following PowerShell commands provide direct HTTP checks without a browser:

```

curl.exe -i http://127.0.0.1:8888/
curl.exe -I http://127.0.0.1:8888/index.html
curl.exe -i -X POST -d "a=6&b=7&op=mul" `
    http://127.0.0.1:8888/cgi-bin/calculator.py
curl.exe -i -X POST -d "student_id=1120231863" `
    http://127.0.0.1:8888/cgi-bin/query.py

```


4.3. Windows Distribution

PyInstaller creates a self-contained onedir distribution. This mode contains the executable together with its Python runtime, imported modules, Web resources, and instructions [9]. The build command is:

```
.\.venv\Scripts\pyinstaller.exe --clean --noconfirm `
1120231863CGIMultiThreadedWebServer.spec
```

The generated directory contains 1120231863CGIMultiThreadedWebServer.exe, _internal, webroot, and README.txt. These items must remain together. On Windows 10 or later, the program can be started from PowerShell without installing Python:

```
.\1120231863CGIMultiThreadedWebServer.exe
```

The current distribution occupies approximately 18.78 MiB. A native program for another operating system must be built on that operating system because the package is platform-specific.

5. System Test

5.1. Test Strategy and Environment

The tests were run on Windows 10 with Python 3.13.3. Unit tests check individual parsers, routers, CGI functions, connection operations, and logging behavior. Integration tests start the real TCP server on an unused local port and communicate through sockets.

The suite contains 58 tests. The following excerpt shows three protocol cases and the 40-client concurrency case. These tests check both response values and connection behavior.

```
def test_rejects_post_without_content_length(self):
    reader = self.make_reader([b"POST /submit HTTP/1.0\r\n\r\n"])
    error_type = getattr(http, "HttpRequestError", Exception)
    with self.assertRaises(error_type):
        reader.read_request()

def test_head_serialization_omits_body_but_keeps_length(self):
    request = http.HttpRequest("HEAD", "/index.html", "HTTP/1.0", {}, b"")
    response = http.HttpResponse(200, {"Content-Type": "text/plain"}, b"hello")
    wire = http.serialize_response(response, request, keep_alive=False)
    head, body = wire.split(b"\r\n\r\n", 1)
    self.assertIn(b"Content-Length: 5", head)
    self.assertEqual(b"", body)

def test_forty_concurrent_requests_are_correct(self):
    self.start(workers=4, max_connections=64)
    def fetch(_):
        status, _, body = self.request("GET", "/index.html")
        return status, body
    with ThreadPoolExecutor(max_workers=40) as pool:
        results = list(pool.map(fetch, range(40)))
    self.assertTrue(all(status == 200 for status, _ in results))
```

5.2. Automatic Test Results

Category	Cases	Main checks	Result
HTTP parsing and response	12	Fragmentation, buffering, lengths, limits, keep-alive, and HEAD.	Pass
Static routing	8	Index, MIME, custom 404, protected paths, and traversal.	Pass
CGI executor	5	Environment, stdin, status, timeout, malformed output, and allow-list.	Pass
CGI applications	9	Form parsing, arithmetic, database creation, query, and injection input.	Pass
Connection manager	6	Queue, oldest eviction, release, and shutdown wake-up.	Pass
Server integration	7	GET/HEAD, CGI, errors, persistence, 40 clients, and eviction.	Pass
Logging, CLI, as-sets, and tools	11	Concurrent log lines, defaults, required files, statistics, and auxiliary tools.	Pass
Total	58	Complete automatic suite	Pass

Table 6: Summary of the automatic test suite.

TERMINAL SCREENSHOT PLACEHOLDER — automatic tests

Run in PowerShell:

```
.\.venv\Scripts\python.exe -W error::ResourceWarning -m unittest discover -s tests -p "test_*.py" -v
```

Capture the final lines showing Ran 58 tests and OK, together with several test names above them.

The complete suite finished with all 58 tests passing. Resource warnings were treated as errors, so a leaked socket or file handle would also fail the run.

5.3. Browser and CGI Tests

The home page was opened in a browser at the default address. The page loaded its style sheet and SVG asset through separate HTTP requests. The result is shown in Figure 3.



Figure 3: Static home page returned by the server.

For the calculator test, the form submitted 6, 7, and multiplication by POST. The CGI process returned the result 42, as shown in Figure 4.

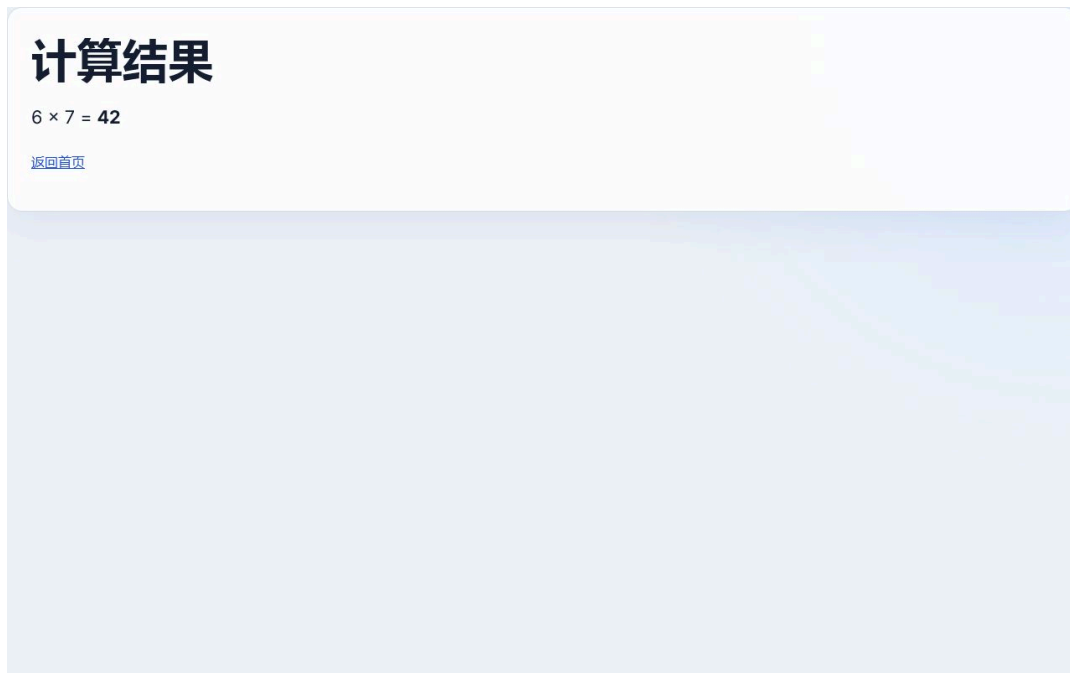


Figure 4: Dynamic calculator result returned by CGI.

For the query test, the submitted student ID was read from the form body. The query CGI opened the SQLite database and returned the matching ID, name, and class in Figure 5.



Figure 5: Dynamic student information returned by CGI.

5.4. Robustness Tests

The robustness tests include percent-encoded directory traversal, protected directories, an unlisted CGI path, CGI timeout, malformed CGI output, invalid calculator input, and an SQL-injection string. The expected result is a controlled 400, 403, or 500 response without exposing files or executing unauthorized code.

6. Performance and Analysis

6.1. Experiment Setup

The performance experiment used the static `/index.html` page. Each measured request opened a new TCP connection and sent an HTTP/1.0 GET request. This workload emphasizes connection admission, worker scheduling, HTTP parsing, file reading, and response transmission. It does not measure CGI or wide-area network performance.

The worker count was set to 1, 2, 4, 8, 12, 16, and 20. For every setting, the tool sent 500 measured requests with a client concurrency of 40. It first sent 20 warm-up requests and repeated the measured run three times. The summary uses the median of the three repetitions, which reduces the effect of a single short disturbance.

```
.\.venv\Scripts\python.exe tools\performance_suite.py `
--workers 1 2 4 8 12 16 20 `
--requests 500 `
--concurrency 40 `
--repetitions 3 `
--warmup 20 `
--output-dir results
```

Throughput is the number of successful responses divided by the measured duration. Mean, median, and P95 latency are calculated from individual request times. The response status and body are checked before a request is counted as successful.

6.2. Results

Work-ers	Throughput	Mean	Median	P95	Errors
1	458.42 req/s	82.81 ms	80.81 ms	104.36 ms	0
2	644.06 req/s	58.10 ms	57.85 ms	66.77 ms	0
4	815.43 req/s	46.18 ms	47.20 ms	54.94 ms	0
8	782.26 req/s	48.59 ms	50.35 ms	54.04 ms	0
12	747.50 req/s	50.29 ms	51.30 ms	61.85 ms	0
16	650.87 req/s	57.77 ms	58.30 ms	69.70 ms	0
20	648.41 req/s	58.77 ms	59.14 ms	70.25 ms	0

Table 7: Median performance results from three repetitions.

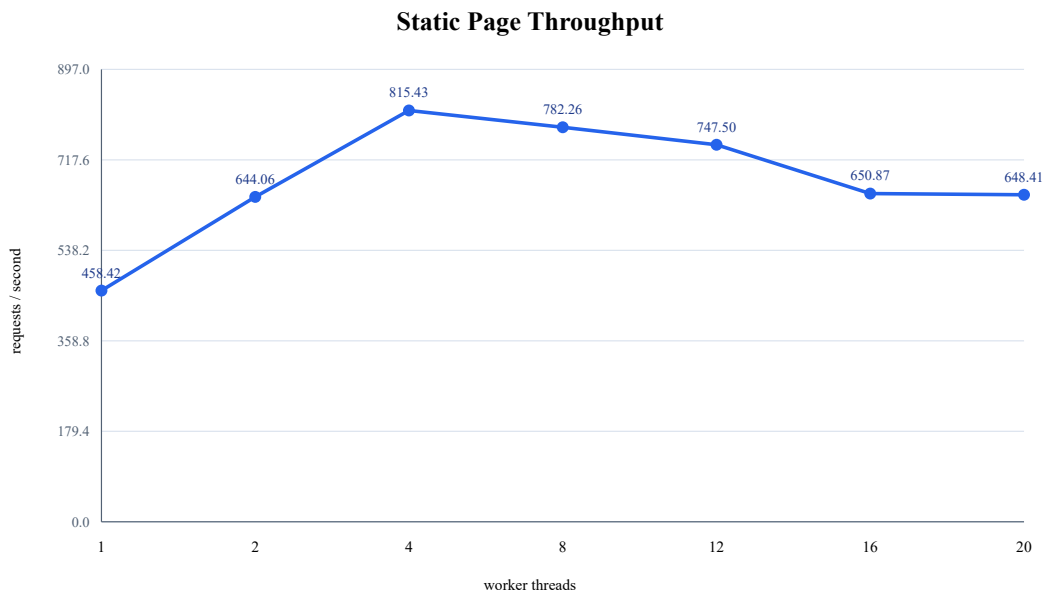


Figure 6: Static-page throughput at different worker counts.

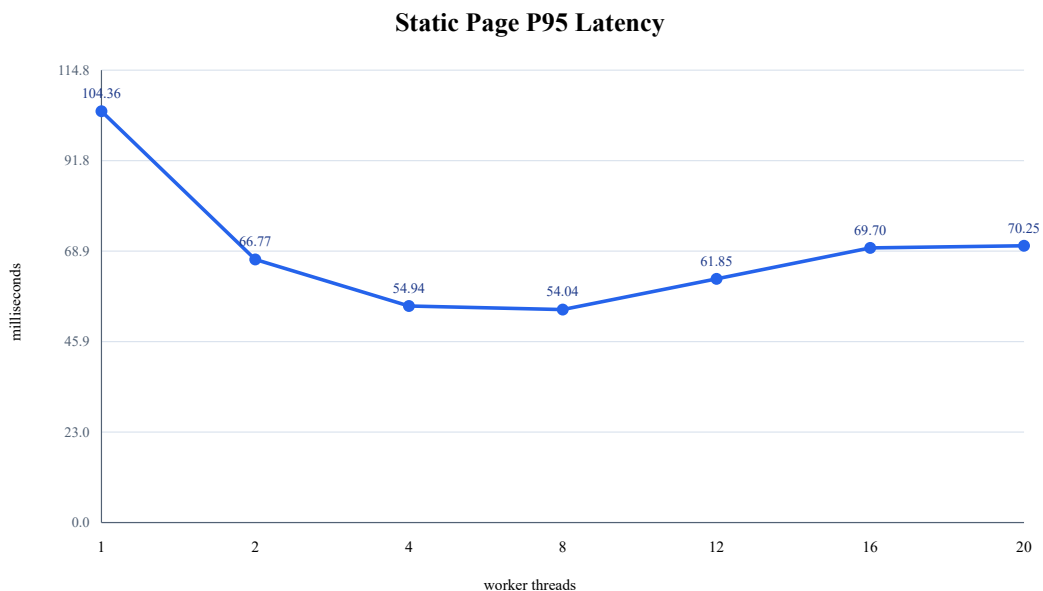


Figure 7: P95 response latency at different worker counts.

The largest improvement occurred between one and four workers. Throughput increased from 458.42 to 815.43 requests per second, while mean latency decreased from 82.81 to 46.18 ms. Four workers gave the highest measured throughput.

Adding more workers did not produce a monotonic improvement. At eight workers the throughput was still 782.26 requests per second, but it gradually fell to 648.41 at 20 workers. All tests ran on one computer, and every request created a new connection. After enough I/O waits were overlapped, extra scheduling and TCP connection overhead became more visible. The experiment therefore suggests that four to eight workers are suitable for this local workload. It does not establish a universal best value for other computers or CGI workloads.

Across the seven worker settings and three repetitions, all 10,500 measured requests returned the expected response and the recorded error count remained zero.

7. Summary or Conclusions

This project implemented a CGI-support multi-threaded Web server using low-level TCP sockets. The server parses HTTP/1.0 requests, supports GET, POST, and HEAD, serves static resources, and returns

dynamic results from two CGI subprocesses. A fixed worker pool handles queued connections, while a separate connection manager enforces the open-connection limit and closes the earliest accepted connection when necessary.

The implementation also includes controlled error responses, safe document-root path resolution, an allow-list for CGI programs, CGI time and output limits, a parameterized SQLite query, and a thread-safe access log. The Windows distribution can run without a separately installed Python environment when its complete directory is kept together.

All 58 automatic tests passed, including real socket tests, 40 concurrent clients, connection eviction, CGI execution, path traversal, logging, and graceful shutdown. In the local static-page experiment, the highest throughput was 815.43 requests per second with four workers, and all 10,500 measured requests succeeded.

The server remains a course-scale implementation. It does not implement TLS, chunked transfer encoding, caching, range requests, authentication, file upload, or unrestricted CGI execution. These boundaries kept the design focused on socket programming, HTTP message processing, thread coordination, and CGI data exchange.

8. References

- [1] T. Berners-Lee, R. Fielding, and H. Frystyk, “Hypertext Transfer Protocol – HTTP/1.0,” RFC 1945, May 1996. <https://www.rfc-editor.org/rfc/rfc1945.html>
- [2] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” RFC 9110, June 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>
- [3] D. Robinson and K. Coar, “The Common Gateway Interface (CGI) Version 1.1,” RFC 3875, October 2004. <https://www.rfc-editor.org/rfc/rfc3875.html>
- [4] Python Software Foundation, “socket — Low-level networking interface,” Python 3.13 documentation. <https://docs.python.org/3.13/library/socket.html>
- [5] Python Software Foundation, “threading — Thread-based parallelism,” Python 3.13 documentation. <https://docs.python.org/3.13/library/threading.html>
- [6] Python Software Foundation, “subprocess — Subprocess management,” Python 3.13 documentation. <https://docs.python.org/3.13/library/subprocess.html>
- [7] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python 3.13 documentation. <https://docs.python.org/3.13/library/sqlite3.html>
- [8] OWASP Foundation, “Path Traversal.” https://owasp.org/www-community/attacks/Path_Traversal
- [9] PyInstaller Development Team, “How the One-Folder Program Works,” PyInstaller documentation. <https://pyinstaller.org/en/stable/operating-mode.html>

9. Comments

This project connects several topics that are usually introduced separately, including TCP sockets, HTTP, threads, subprocesses, and databases. Implementing the HTTP parser directly made the relation between a TCP byte stream and an application-layer message much clearer. The performance experiment was also useful because it showed that increasing the thread count does not always improve a server on the same workload.

For a future version of the project, a short protocol-conformance checklist and a fixed performance-test format would make results from different implementations easier to compare. A cross-host test could also check server accessibility on a real local network and show how firewall settings and concurrent connections affect the result.